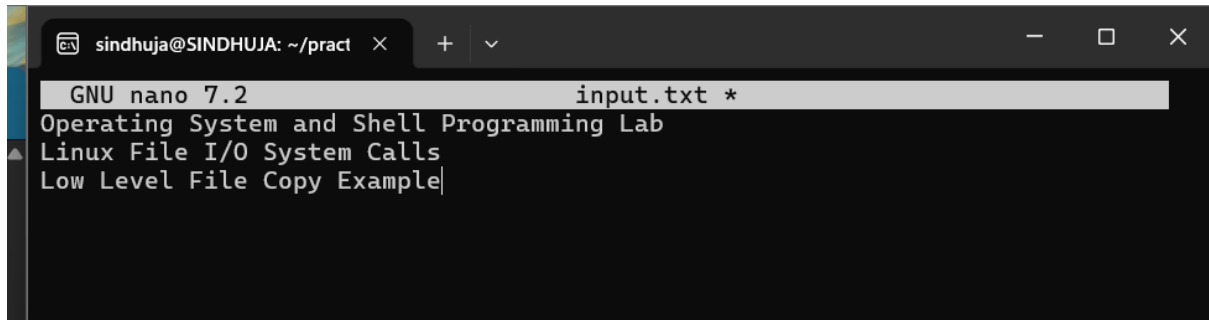


PRACTICAL WEEK 9

Aim:

To implement a file copy utility using low-level Linux I/O system calls (open, read, write, lseek, close) and compare its performance with standard library functions (fopen, fread, fwrite).



```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 input.txt *
Operating System and Shell Programming Lab
Linux File I/O System Calls
Low Level File Copy Example
```



```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 practical9_task1.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("input.txt", O_RDONLY);

    if (source < 0)
    {
        printf("Error opening source file.\n");
        return 1;
    }

    destination = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        printf("Error creating destination file.\n");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

    lseek(source, 0, SEEK_SET);

    printf("File copied successfully using low-level I/O system calls.\n");

    close(source);
    close(destination);

    return 0;
}
File to insert [from ./]:
^G Help          M-F New Buffer    ^X Execute Command
^C Cancel        M-N No Conversion ^T Browse
```

```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 practical9_compare.c *
#include <stdio.h>

int main()
{
    FILE *source, *destination;
    char buffer[100];

    source = fopen("input.txt", "r");
    destination = fopen("output_library.txt", "w");

    if (source == NULL || destination == NULL)
    {
        printf("Error opening file.\n");
        return 1;
    }

    while (fgets(buffer, sizeof(buffer), source) != NULL)
    {
        fputs(buffer, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
}
```

Output:

```
sindhuja@SINDHUJA: ~/pract x + v
sindhuja@SINDHUJA:~$ cd practical_week9
sindhuja@SINDHUJA:~/practical_week9$ nano input.txt
sindhuja@SINDHUJA:~/practical_week9$ nano practical9_task1.c
sindhuja@SINDHUJA:~/practical_week9$ gcc practical9_task1.c -o practical9_ta
sk1
sindhuja@SINDHUJA:~/practical_week9$ ./practical9_task1
File copied successfully using low-level I/O system calls.
sindhuja@SINDHUJA:~/practical_week9$ cat output.txt
Operating System and Shell Programming Lab
Linux File I/O System Calls
Low Level File Copy Example
sindhuja@SINDHUJA:~/practical_week9$ nano practical9_compare.c
sindhuja@SINDHUJA:~/practical_week9$ gcc practical9_compare.c -o practical9_
compare
sindhuja@SINDHUJA:~/practical_week9$ ./practical9_compare
File copied successfully using standard library functions.
sindhuja@SINDHUJA:~/practical_week9$ cat output_library.txt
Operating System and Shell Programming Lab
Linux File I/O System Calls
Low Level File Copy Example
sindhuja@SINDHUJA:~/practical_week9$ ~|
```

Aim:

To develop a C program that redirects standard input and standard output using file descriptors and dup2(), and demonstrate input/output redirection in Linux.

```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 input_redirect.txt *
This is OSSP Practical Week 9.
Input redirection using dup2().
Output redirection using dup2().|
```

```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 practical9_task2.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int inputFile, outputFile;
    char buffer[100];
    int bytesRead;

    inputFile = open("input_redirect.txt", O_RDONLY);
    outputFile = open("output_redirect.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);

    if (inputFile < 0 || outputFile < 0)
    {
        printf("Error opening files.\n");
        return 1;
    }

    dup2(inputFile, STDIN_FILENO);
    dup2(outputFile, STDOUT_FILENO);

    while ((bytesRead = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
    {
        write(STDOUT_FILENO, buffer, bytesRead);
    }

    close(inputFile);
    close(outputFile);

    return 0;
}|

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```

sindhuja@SINDHUJA:~/practical_week9$ nano input_redirect.txt
sindhuja@SINDHUJA:~/practical_week9$ nano practical9_task2.c
sindhuja@SINDHUJA:~/practical_week9$ gcc practical9_task2.c -o practical9_task2
sindhuja@SINDHUJA:~/practical_week9$ ./practical9_task2
sindhuja@SINDHUJA:~/practical_week9$ cat output_redirect.txt
This is OSSP Practical Week 9.
Input redirection using dup2().
Output redirection using dup2().
sindhuja@SINDHUJA:~/practical_week9$ cat < input_redirect.txt > shell_output.txt
sindhuja@SINDHUJA:~/practical_week9$ cat shell_output.txt
This is OSSP Practical Week 9.
Input redirection using dup2().
Output redirection using dup2().
sindhuja@SINDHUJA:~/practical_week9$ |

```

Observation:

1. The dup2() system call redirected standard input and output.
2. File descriptors STDIN_FILENO and STDOUT_FILENO were replaced.
3. The program copied data from one file to another through redirected file descriptors.
4. Shell redirection using < and > produced the same result.
5. Input/output redirection was successfully demonstrated

Result:

Successfully implemented standard input and output redirection using dup2() and demonstrated shell I/O redirection using Linux file descriptors.